Most Commonly Asked Coding Q/A

1. Reverse a String without using Inbuilt method (Important)

Input : India is my Country

```
String input = "India is my Country";
String reversed = "";
for (int i = input.length() - 1; i >= 0; i--) {
   reversed = reversed + input.charAt(i);
}
System.out.println("Reversed String: " + reversed);
```

Output :

Reversed String: yrtnuoC ym si aidnI

2. Count Occurrence of Each Character (Important)

Input : India is my Country

```
String input = "India is my Country";
 HashMap<Character, Integer> map = new LinkedHashMap<>();
      for (char ch : input.toCharArray()) {
         if (map.containsKey(ch)) {
            map.put(ch, map.get(ch) + 1);
         } else {
            map.put(ch, 1);
         }
      }
      System.out.println(map);
```

Output :

{I=1, n=2, d=1, i=2, a=1, =3, s=1, m=1, y=2, C=1, o=1, u=1, t=1, r=1}

3. Print only the Unique value (Important)

Input : India is my Country

```
String input = "India is my Country";
 HashMap<Character, Integer> map = new LinkedHashMap<>();
      for (char ch : input.toCharArray()) {
         map.put(ch, map.getOrDefault(ch, 0) + 1);
      }
      for (char ch : input.toCharArray()) {
         if (map.get(ch) == 1) {
            System.out.print(ch + " ");
   }
}
```

Output : I d a s m C o u t r

4. Remove Duplicate from Array (Important)

Input : {2,7,8,10,4,8,7,10};

```
   int a[] = {2, 7, 8, 10, 4, 8, 7, 10};
    HashSet<Integer> set = new LinkedHashSet<>();
      for (int num : a) {
         set.add(num);
      }
      System.out.println("Array after removing duplicates: " + set);
```

Output :

Array after removing duplicates: [2, 7, 8, 10, 4]

5. Arrange the array in the Ascending order without using inbuilt method (Important)

Input : {2,7,8,10,4,8,7,10};

```
int a[] = {2, 7, 8, 10, 4, 8, 7, 10};
    for (int i = 0; i < a.length - 1; i++) {
       for (int j = i+1; j < a.length; j++) {
          if (a[i] > a[j]) {
             int temp = a[i];
             a[i] = a[j];
             a[j] = temp;
          }
       }
    }
    for (int num : a) {
       System.out.print(num + " ");
    }
```

Output : 2 4 7 7 8 8 10 10

6. Find whether the given Integer is Prime number or not (Important)

Input : Num = 29

```
    int num = 29;
    boolean isPrime = true;
    if (num <= 1) {
       isPrime = false;
    } else {
       for (int i = 2; i <= Math.sqrt(num); i++) {
          if (num % i == 0) {
             isPrime = false;
```

```
            break;
        }
      }
    }
    if (isPrime){
      System.out.println(num + " is a Prime Number");
}
    else{
      System.out.println(num + " is NOT a Prime Number");
}
```

7. Arrange the array in the Descending order without using inbuilt method (Important)

Input : {2,7,8,10,4,8,7,10};

```
int a[] = {2, 7, 8, 10, 4, 8, 7, 10};
    for (int i = 0; i < a.length - 1; i++) {
      for (int j = i+1; j < a.length; j++) {
        if (a[i] < a[j]) {
          int temp = a[i];
          a[i] = a[j];
          a[j] = temp;
        }
      }
    }
    for (int num : a) {
      System.out.print(num + " ");
    }
```

Output : 10 10 8 8 7 7 4 2

8. Reverse the array (Important)

Input : {2,7,8,10,4,8,7,10};

```
int a[] = {2, 7, 8, 10, 4, 8, 7, 10};
        int start = 0;
        int end = a.length - 1;
        while (start < end) {
            int temp = a[start];
            a[start] = a[end];
            a[end] = temp;
            start++;
            end--;
        }
        for (int num : a) {
            System.out.print(num + " ");
        }
```

Output : 10 7 8 4 10 8 7 2

9. Find the largest number in an array. (Important)

Input : {2,7,8,10,4,8,7,10};

```
       int a[] = {2, 7, 8, 10, 4, 8, 7, 10};
        int largest = a[0];
        for (int i = 1; i < a.length; i++) {
            if (a[i] > largest) {
                largest = a[i];
            }
        }
 System.out.println("Largest Number: " + largest);
```

10. Find the second largest number in an array. (Important)

Input : {2,7,8,10,4,8,7,10};

```
int a[] = {2, 7, 8, 10, 4, 8, 7, 10};
int largest = Integer.MIN_VALUE;
int secondLargest = Integer.MIN_VALUE;
for (int i = 0; i < a.length; i++) {
   if (a[i] > largest) {
      secondLargest = largest;
      largest = a[i];
   }
   else if (a[i] > secondLargest && a[i] != largest) {
      secondLargest = a[i];
   }
}
System.out.println(secondLargest);
```

Output : 8

11. Swap two numbers without using a temporary variable

```
   int a = 10;
    int b = 20;
a = a + b;
b = a - b;
a = a – b;
System.out.println("After Swap: a = " + a + ", b = " + b);
```

12. Reverse an integer

```
int num = 12345;
int reversed = 0;
while (num != 0) {
   int digit = num % 10
   reversed = reversed * 10 + digit;
   num = num / 10;
}
System.out.println("Reversed Number: " + reversed);
```

Output : Reversed Number: 54321

13. Factorial of a number

```
int num = 5; // Change number here
long factorial = 1;
for (int i = 1; i <= num; i++) {
   factorial *= i;
}
System.out.println("Factorial of " + num + " is " + factorial);
```

14. Increase each character in a string by 1 letter

```
String str = "ABC";
StringBuilder result = new StringBuilder();
for (int i = 0; i < str.length(); i++) {
   char ch = str.charAt(i);
   ch = (char)(ch + 1);
   result.append(ch);
}
System.out.println("Incremented String: " + result.toString());
```

15. Find the second largest word in a string (Important)

```
String str = "India is my Country";
String[] words = str.split(" ");
String largest = "";
String secondLargest = "";
for (String word : words) {
   if (word.length() > largest.length()) {
      secondLargest = largest;
      largest = word;
   } else if (word.length() > secondLargest.length() &&
!word.equals(largest)) {
      secondLargest = word;
   }
}
System.out.println("Second Largest Word: " + secondLargest);
```

16. Move all zeros to the end of an array without changing the order

Input : {2, 0, 8, 0, 4, 0, 7}

Output : {2 8 4 7 0 0 0}

```
int a[] = {2, 0, 8, 0, 4, 0, 7};
int number = 0;
for (int i = 0; i < a.length; i++) {
   if (a[i] != 0) {
      int temp = a[number];
      a[number] = a[i];
      a[i] = temp;
      number++;
   }
```

```
}
for (int num : a) {
   System.out.print(num + " ");
}
```

17. **Add all the numbers in a string (Important)**

```
String str = "dh47dg6jd39jefh8df";
int sum = 0;
// Regex to match one or more digits
Pattern p = Pattern.compile("\\d+");
Matcher m = p.matcher(str);
while (m.find()) {
   String number = m.group();         // Extract the number
   sum += Integer.parseInt(number);    // Convert and add
}
System.out.println("Sum of numbers in the string: " + sum);
```

Output : 100

18. Left-Angled Triangle

```
for (int i = 1; i <= 5; i++) {
  for (int j = 1; j <= i; j++) {
     System.out.print("*");
          }
   System.out.println();
}
```

Output:

```
*
**
***
****
*****
```

19. Right-Angled Triangle

```
for (int i = 1; i <= 5; i++) {
   for (int j = 1; j <= 5 - i; j++) {
         System.out.print(" ");
         }
   for (int k = 1; k <= i; k++){
          System.out.print("*");
         }
   System.out.println();
}
```

Output:

```
    *
   **
  ***
 ****
*****
```

20. Pyramid pattern

```
  for (int i = 1; i <= 5; i++) {
   for (int j = 1; j <= 5 - i; j++) {
      System.out.print(" ");
   }
```

```
    for (int k = 1; k <= 2 * i - 1; k++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Output:

```
    *
   ***
  *****
 *******
*********
```

21. Increasing & Decreasing Pattern

```
        int rows = 5;
        for (int i = 1; i <= rows; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        for (int i = rows - 1; i >= 1; i--) {
            for (int j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
*
```

**

***

****

*****

****

***

**

*

24. check whether a String is Palindrome

```
String str = "madam";
String reversed = "";
for (int i = str.length() - 1; i >= 0; i--) {
   reversed += str.charAt(i);
}
if (str.equals(reversed)){
   System.out.println("Palindrome");
}
else{
   System.out.println("Not Palindrome");
}
```

25. First non-repeated character in a String (Important)

```
String str = "automation";
Map<Character, Integer> map = new LinkedHashMap<>();
for (char ch : str.toCharArray()) {
   map.put(ch, map.getOrDefault(ch, 0) + 1);
}
```

```
for (Map.Entry<Character, Integer> entry : map.entrySet()) {
    if (entry.getValue() == 1) {
        System.out.println("First non-repeated character: " + entry.getKey());
        break;
    }
}
```

Output :

First non-repeated character: u

27. reverse words in a sentence (Important)

```
String str = "Java is powerful";
String[] words = str.split(" ");
for (int i = words.length - 1; i >= 0; i--) {
    System.out.print(words[i] + " ");
}
```

Output : powerful is Java

28. Diamond Pattern

```
int rows = 5;
for (int i = 1; i <= rows; i++) {
    for (int j = 1; j <= rows - i; j++) {
        System.out.print(" ");
    }
    for (int k = 1; k <= 2 * i - 1; k++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
        // Lower Inverted Pyramid
        for (int i = rows - 1; i >= 1; i--) {
            for (int j = 1; j <= rows - i; j++) {
                System.out.print(" ");
            }
            for (int k = 1; k <= 2 * i - 1; k++) {
                System.out.print("*");
            }
            System.out.println();
        }
```

Output:

```
   *
  ***
 *****
*******
 *******
  *****
   ***
    *
```

29. Merge two arrays

```
        int a[] = {1, 3, 5};
        int b[] = {2, 4, 6};
        int merged[] = new int[a.length + b.length];
        // Copy first array
        for (int i = 0; i < a.length; i++) {
```

```
        merged[i] = a[i];
    }
    // Copy second array
    for (int i = 0; i < b.length; i++) {
        merged[a.length + i] = b[i];
    }
    // Print merged array
    for (int num : merged) {
        System.out.print(num + " ");
    }
```

Output: (1 3 5 2 4 6}

30. Anagram check

```
    String str1 = "listen";
    String str2 = "silent";
    // Remove spaces and convert to lowercase
    str1 = str1.replaceAll("\\s", "").toLowerCase();
    str2 = str2.replaceAll("\\s", "").toLowerCase();
    if (str1.length() != str2.length()) {
        System.out.println("Not an Anagram");
        return;
    }
    char[] arr1 = str1.toCharArray();
    char[] arr2 = str2.toCharArray();
    Arrays.sort(arr1);
    Arrays.sort(arr2);
    if (Arrays.equals(arr1, arr2)){
```

```
        System.out.println("Anagram");
}
    else{
        System.out.println("Not an Anagram");
}
```

31. Armstrong Number

153 → $1^3 + 5^3 + 3^3 = 153$ ✅

9474 → $9^4 + 4^4 + 7^4 + 4^4 = 9474$ ✅

123 → $1^3 + 2^3 + 3^3 = 36$ ❌

```
    int num = 153; // Number to check
    int originalNum = num;
    int sum = 0;
    // Count digits
    int digits = String.valueOf(num).length();
    while (num > 0) {
        int remainder = num % 10;
        sum += Math.pow(remainder, digits);
        num /= 10;
    }
    if (sum == originalNum)
        System.out.println(originalNum + " is an Armstrong number");
    else
        System.out.println(originalNum + " is NOT an Armstrong number");
```

32. print a Fibonacci series

The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding ones, starting from 0 and 1.

Example:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34

```
int n = 10; // Number of terms
 int first = 0;
 int second = 1;
 for (int i = 3; i <= n; i++) {
    int next = first + second;
    System.out.print(next + " ");
    first = second;
    second = next;
 }
```

Output : 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

33. Find missing number

```
int arr[] = {1, 2, 4, 5, 6};
 int n = arr.length + 1; // Because one number is missing
 int total = n * (n + 1) / 2; // Sum of 1 to n
 int sum = 0;
 for (int num : arr) {
    sum += num;
 }
 int missing = total - sum;
 System.out.println("Missing number is: " + missing);
```

Output: Missing number is: 3

34. Reverse the characters in each word

Input: "Java is powerful"

Output: "avaJ si lufrewop"

```
String str = "Java is powerful";
String[] words = str.split(" ");
String result = "";
for (String word : words) {
    String rev = "";
    for (int i = word.length() - 1; i >= 0; i--) {
        rev += word.charAt(i);
    }
    result += rev + " ";
}
System.out.println(result.trim());
```

35.Reverse only the letters in the string , keeping the positions of digits or special characters intact.

Input: "2abcd3"

Output: "2dcba3"

```
String str = "2abcd3";
char[] arr = str.toCharArray();
int left = 0;
int right = arr.length - 1;
while (left < right) {
    if (!Character.isLetter(arr[left])) {
        left++;
    }
```

```
        else if (!Character.isLetter(arr[right])) {
            right--;
        }
        else {
            char temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }
    System.out.println(new String(arr));
```

36. Remove vowels from a string

Input: "Automation"

Output: "tmtn"

```
    String str = "Automation";
    String vowels = "aeiouAEIOU";
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        if (vowels.indexOf(ch) == -1) {
            sb.append(ch);
        }
    }
    System.out.println(sb.toString());
```

37. Reverse only the vowels in a string

Input: "hello"

Output: "holle"

```
String str = "hello";
 char[] arr = str.toCharArray();
 String vowels = "aeiouAEIOU";
 int left = 0;
 int right = arr.length - 1;
 while (left < right) {
    if (vowels.indexOf(arr[left]) == -1) {
       left++;
    }
    else if (vowels.indexOf(arr[right]) == -1) {
       right--;
    }
    else {
       char temp = arr[left];
       arr[left] = arr[right];
       arr[right] = temp;
       left++;
       right--;
    }
 }
 System.out.println(new String(arr));
```

38. Replace all vowels with 'X'

Input: "Automation"

Output: "XXtXmXtXXn"

```
String str = "Automation";
String vowels = "aeiouAEIOU";
StringBuilder sb = new StringBuilder();
for (int i = 0; i < str.length(); i++) {
   char ch = str.charAt(i);
   if (vowels.indexOf(ch) != -1) {
      sb.append('X'); // replace vowel with X
   } else {
      sb.append(ch);
   }
}
System.out.println(sb.toString());
```

39. Print only the duplicate characters in a string

```
String str = "India is my Country";
str =str .toLowerCase(); // case-insensitive
str = str.replaceAll("\\s", ""); // remove spaces
Map<Character, Integer> map = new LinkedHashMap();
// Count frequency of each character
for (char ch : str.toCharArray()) {
   map.put(ch, map.getOrDefault(ch, 0) + 1);
}
for (Map.Entry<Character, Integer> entry : map.entrySet()) {
   if (entry.getValue() > 1) {
```

```
            System.out.print(entry.getKey() + " ");
        }
    }
```

Output : i n y

40. Remove duplicate characters from a string

Input: "India is my Country"

Output: "Inda smyCoutr"

```
String str = "India is my Country";
    Set<Character> set = new LinkedHashSet<>();
    StringBuilder sb = new StringBuilder();
    for (char ch : str.toCharArray()) {
        if (set.add(ch)) {  // Returns false if character is already present
            sb.append(ch);
        }
    }
    System.out.println(sb.toString());
```